

Intoxalock Device Removal — Team Ownership & AWS Architecture PRD

Product Requirements Document derived from the "Team Ownership & Contracts" and "AWS Architecture" solution diagrams.

This document specifies two facets of the automated Device Removal solution: (1) the **team ownership model** that divides responsibility between Intoxalock and Source Meridian and the **integration contracts** that bind the two zones together, and (2) the **AWS serverless architecture** — an AWS Step Functions Standard workflow where one execution equals one removal request. It is the authoritative reference for what each team builds, the exact request/response payloads exchanged across team boundaries, and the state machine that orchestrates automated appointment confirmation.

1. Overview

The Device Removal solution automates the end-to-end process by which a customer requests removal of an Intoxalock device, an appointment is confirmed with an independent Service Center, and a Work Order is issued — with a human representative involved only on exceptions. The work is split across two organizations, so clear ownership boundaries and versioned integration contracts are essential.

In scope for this PRD. The two team zones and their responsibilities; the three inter-team integration contracts (C1, C2, C3); and the AWS architecture implementing the Automated Confirmation engine, including the Step Functions state machine, the reusable Outbound Call Service, data stores, and the shared escalation path.

Out of scope. The end-to-end customer sequence (covered in the Sequence diagram), UI/UX for intake channels, and the internal implementation of Intoxalock's backend APIs (owned by Intoxalock).

2. Actors

Actor	Zone	Role
Customer	External	Requests device removal; receives acknowledgment, callbacks, and appointment confirmation.
Service Center	External	Independent provider that confirms the appointment time and quote, or declines with alternatives.
Intoxalock Representative	External	Steps in only on exceptions, working from a ticket that carries the full interaction history.
Intoxalock team	Intoxalock	Owns Request Intake (Phase 1), Work-Order issuance, and the human-escalation ticketing surface.
Source Meridian	Source Meridian	Owns the Automated Confirmation engine — orchestration, AI voice agents, and escalation triggering.

3. Team Ownership Model

Ownership is divided into two zones. Everything that crosses a zone boundary must travel over one of the numbered integration contracts (C1–C3).

3.1 Intoxalock team — Request Intake + Work-Order issuance

Component	Responsibility
Request Intake	Runs the Eligibility Gate; accepts requests via channels (App / IVR); captures the customer's available time slots.
Work Order Creation + Appointment Confirmation	Receives the confirmed appointment → generates the Work Order → notifies the customer that it is "ready".
Ticketing System	Human escalation queue. <i>Open question:</i> will this be email / Slack / Zendesk?

3.2 Source Meridian — Automated Confirmation engine

Component	Responsibility
Confirmation Orchestrator	Drives the automated confirmation flow and decides when to retry, call back, or escalate.
Service-Center AI Agent	AI voice agent that calls the Service Center to confirm the appointment; retries on no-answer.
Customer Callback AI Agent	AI voice agent that calls the customer to confirm a new date on the decline path; retries.
Human Agent Escalation	Fires when the request cannot be resolved automatically and must be handed to a representative.

3.3 Ownership handoff flow

1. **Submit Removal Request (App / IVR)** — Customer → Request Intake (*Intoxalock*).
2. **C1 · Start Automated Confirmation (webhook)** — Request Intake → Confirmation Orchestrator (*crosses to Source Meridian*).
3. **AI voice** — Service-Center AI Agent → Service Center; the center replies to **confirm time + quote** or **decline + alternative times**.
4. **Decline path** — Orchestrator → Customer Callback AI Agent → Customer (AI voice, confirm new date).
5. **C2 · Appointment Confirmed** — Service-Center or Customer Callback agent → Work Order Creation (*crosses back to Intoxalock*) → **appointment ready (SMS / email)** to Customer.
6. **C3 · Escalation / Exception** — Human Agent Escalation → Ticketing System (*crosses back to Intoxalock*) → **ticket + history** → Intoxalock Representative.

4. Integration Contracts

Three contracts govern every zone crossing. All payloads are JSON over HTTPS.

C1 · Start Automated Confirmation (webhook)

Trigger: Intake passes the Eligibility Gate.

Request

```
{
  "customer": {
    "id": "string", "phone": "E.164", "email": "string",
    "name": "string"
  },
  "assigned_service_center": {
    "id": "string", "phone": "E.164", "name": "string", "timezone": "string"
  },
  "customer_preferred_time_slots": ["string", "string"],
  "vehicle": { "make": "string", "model": "string", "year": 2020 },
  "eligibility": true
}
```

Response — 202

```
{ "request_id": "uuid" }
```

C2 · Appointment Confirmed (→ Work Order + notify)

Trigger: Appointment confirmation with quote.

Request

```
{
  "request_id": "uuid",
  "confirmed_date_time": "string",
  "customer_id": "string",
  "service_center_id": "string",
}
```

```

    "quote": { "amount": 0.00 },
    "scheduled_time_slots": ["string"]
  }

```

Response — 200

```

{ "work_order_id": "string" }

```

C3 · Exception / Escalation

Trigger: No agreement reached, or TTL expiry.

Request

```

{
  "requestId": "uuid",
  "customer": {
    "id": "string", "phone": "E.164", "email": "string", "name": "string"
  },
  "assigned_service_center": { "id": "string", "contact": "string", "name": "string" },
  "reason": "NO_AGREEMENT | MAX_SERVICE_CENTER_RETRIES | MAX_CUSTOMER_RETRIES | TTL_EXPIRED",
  "last_state": "string",
  "full_history": [
    { "time_stamp": "string", "channel": "SMS | EMAIL | AI_VOICE_AGENT", "event": "string", "result": "SUCCESS | FAILURE" }
  ]
}

```

Response — 200

```

{ "ticket_id": "string" }

```

5. AWS Architecture

The Automated Confirmation engine is implemented as an **AWS Step Functions Standard workflow — 1 execution = 1 removal request**. Intake happens before the execution starts; the state machine drives confirmation, callback, work-order issuance, reminders, and escalation.

5.1 Volume requirements

Metric	Target
Worst-case burst	50 requests / second
Monthly volume	~5,000 requests / month
Daily volume	~200 requests / day

5.2 Request intake (pre-execution)

Element	Detail
Channels	App form (mobile client); Phone / IVR via Amazon Connect (<i>flagged as possibly out of scope for this phase</i>).
Intoxalock backend	Checks request eligibility for the automated workflow; sends the customer a "request is being processed" confirmation.
Transport	Webhook secured with HMAC (Hash-based Message Authentication Code).
API Gateway (HTTP API)	/intake endpoint.
api-handler Lambda	Auth · validation · computes the timezone-aware center-open target used later by the workflow.

5.3 State machine — happy path and choices

State	Type	Purpose
Start	—	Entry point; one execution per removal request.
Wait Until Center Open	Wait (TimestampPath)	Holds until the center-open target computed at intake.
Enough Time to Confirm?	Choice	now vs latestStartTime; too late → escalate.
Center Confirmation stage	Task + Choice	Voice-first (ElevenLabs + Twilio). Places the outbound center call, then branches on Center Response?
Center Response?	Choice	CONFIRMED → Work Order (skip callback); DECLINED + alt slots → Customer Callback; UNREACHABLE → escalate.
Work Order & Confirm stage	Task	Calls the external Work Order API, then sends confirmed customer SMS / email.
Reminders stage	Wait + Task	Waits until T-12h, then sends the appointment reminder.
Succeed (Confirmed)	Terminal	Successful terminal state.

Center Confirmation stage. Place Outbound Call runs as `startExecution.sync` against the center agent (passing vehicle + slots), with `maxConcurrent=3` and `maxRetries=4`.

5.4 Customer Callback stage (decline path)

Entered when the center declines and offers alternative slots. Governed by the **Slot Hold Window** — how long the center keeps its new slots.

State	Type	Purpose
Set Confirmation Deadline	Pass	<code>deadline = now + slot-hold window</code> , capped at center close.
Wait Until Customer Callable Hours	Wait (TimestampPath)	Customer timezone; TCPA 8am–9pm local ; capped by hold deadline.
Enough Time to Confirm?	Choice	now vs <code>holdDeadline</code> ; too late → escalate.
Place Outbound Call	Task	Customer agent (new date), <code>timeout=holdDeadline</code> , <code>maxConcurrent=3</code> , <code>maxRetries=4</code> .
Customer Response?	Choice	ACCEPTED → Work Order; REJECTED / HOLD_EXPIRED / UNREACHABLE → escalate.

5.5 Outbound Call Service (reusable state machine)

A **separate Standard state machine**, invoked via `startExecution.sync` by both the center and customer call stages. It reserves a line, dials via the voice agent, retries, and returns the outcome. The concurrency cap and retry count are set by the caller.

Input: { agent, payload, callTimeout, maxConcurrent, maxRetries }.

State	Type	Purpose
Try to Reserve a Line (atomic)	DynamoDB	<code>activeCalls +1</code> if under <code>maxConcurrent</code> ; conditional write.
Reserved?	Choice	<code>reserved</code> → Place Call; all lines busy → Wait for a Free Line → retry.
Wait for a Free Line	Wait	Poll interval, then re-attempt the reservation.

State	Type	Purpose
Place Call	Lambda	Dials via the ElevenLabs agent (center or customer, by input) + Twilio; <code>.waitForTaskToken</code> .
Release the Call Line	DynamoDB	<code>activeCalls -1</code> ; also runs on Catch.
Answered?	Choice	<code>connected</code> → <code>outcome</code> → Return; <code>no answer</code> / <code>busy</code> → Retry?
Retry?	Choice	<code>attempts ≤ maxRetries</code> and before <code>callTimeout</code> → Wait Before Retry → re-dial; else give up → UNREACHABLE.
Wait Before Retry	Wait	Configurable retry interval (SSM).
Return	Terminal	Returns <code>{ status, structuredResult }</code> to the caller.

5.6 Data stores and integrations

Resource	Type	Purpose
Removal Requests	DynamoDB	Customer + center info; current appointment state.
Attempts Audit Log	DynamoDB	Append-only per-attempt history; feeds escalation (C3 <code>full_history</code>).
Webhook Handler	Lambda (via API GW)	Idempotent on <code>callId</code> ; resumes the workflow via <code>SendTaskSuccess</code> (task token).
ElevenLabs + Twilio	External voice service	Center Agent and Customer Agent voice calls.
Work Order API	External (Intoxalock)	Called by <code>Create Work Order & Notify</code> (Lambda, with Retry + Catch).
escalate-to-rep API	External (Intoxalock)	Called by <code>Escalate to Human Rep</code> (Lambda, with Retry + Catch).

5.7 Escalation (shared)

Every ESC transition in the diagram converges on the shared escalation stage: **Escalate to Human Rep** (Lambda → `escalate-to-rep API`, with Retry + Catch), which hands off to a human rep via the Intoxalock backend and ends in the **End (Escalated)** terminal state. Escalation triggers include: not enough time to confirm (center or customer), center UNREACHABLE, customer REJECTED / HOLD_EXPIRED / UNREACHABLE, and WO failed.

6. Non-Functional Requirements

Requirement	Detail
Scalability	Sustain ~200 requests/day and ~5,000/month; absorb bursts up to 50 requests/second at intake.
Concurrency control	Outbound calls are gated by an atomic DynamoDB line-reservation counter (<code>maxConcurrent</code> , default 3).
Idempotency	The Webhook Handler is idempotent on <code>callId</code> ; C1 is triggered once per eligible request.
Compliance	Customer callbacks respect TCPA calling hours (8am–9pm customer-local).
Security	Intake webhooks are authenticated with HMAC signatures.

Requirement	Detail
Auditability	Every call attempt is appended to the Attempts Audit Log and surfaced in the C3 escalation payload.
Resilience	External API calls (Work Order, escalate-to-rep) use Step Functions Retry + Catch.

7. Open Questions

- **Ticketing surface** — will human escalation use email, Slack, or Zendesk?
- **IVR scope** — is the Phone / IVR channel (Amazon Connect) in scope for this phase?
- **Slot-hold window** — what is the concrete duration a Service Center guarantees its alternative slots?
- **Retry / concurrency tuning** — confirm `maxConcurrent` and `maxRetries` defaults per call type.